

Danci 背单词 App 本地数据层重构方案

客户端团队技术负责人 · 技术评审会讨论稿 · 2026-08 | Core Data → GRDB.swift 6.x + 自研增量同步

摘要：Q4 整体切换 GRDB + 自研同步

真机实测五项核心指标改善 17–65%、同步冲突全自动解决；Q4 七周完成灰度到全量，请求评审放行。

17–65%

五项核心指标改善
幅度（真机实测）

3.2s

断网 24h 后全量上
行合并（2,300 条
操作）

0.4%

同步冲突率 · 全部
自动解决

7周

Q4 排期：封装 →
灰度 → 全量

现有 Core Data 数据层在离线刷题、多设备同步、SRS 调度的负载下已出现写入卡顿与迁移困难。本方案将其整体替换为 GRDB.swift 6.x + 自研增量同步引擎，并封装 Repository 层隔离第三方依赖。

预研已在真机完成同口径对比与同步 POC：存储侧五项指标全面改善，同步冲突全部按既定策略自动解决，大规模 DAU 场景压测达标——证据见后两节。迁移采用双写灰度，回滚窗口 7 天。

请求评审会放行：Q4 七周排期（第 1–2 周封装 / 第 3–6 周灰度 / 第 7 周全量）与双写灰度方案。

背景：iOS 14 时代的 Core Data 撑不住新形态

离线刷题、多设备同步、SRS 调度加入后，写入卡顿与迁移困难已经不是修补能压住的问题。

数据层搭建于 iOS 14 时代，最初只服务单机词库查询。离线刷题把写入频率抬了一个量级，SRS 调度引入高频复杂查询，多设备同步则要求可靠的本地操作记录——三类新负载全部压在同一套 Core Data 栈上。

写入卡顿直接伤刷题手感，迁移困难则拖慢每一次数据模型演进。在既有约束下评估：继续修补的边际收益递减，整体替换存储框架并引入同步引擎，才能支撑未来两年的功能演进。

约束	取值
系统基线	iOS 16+ 起步
团队技术栈	Swift 为主
架构原则	离线优先

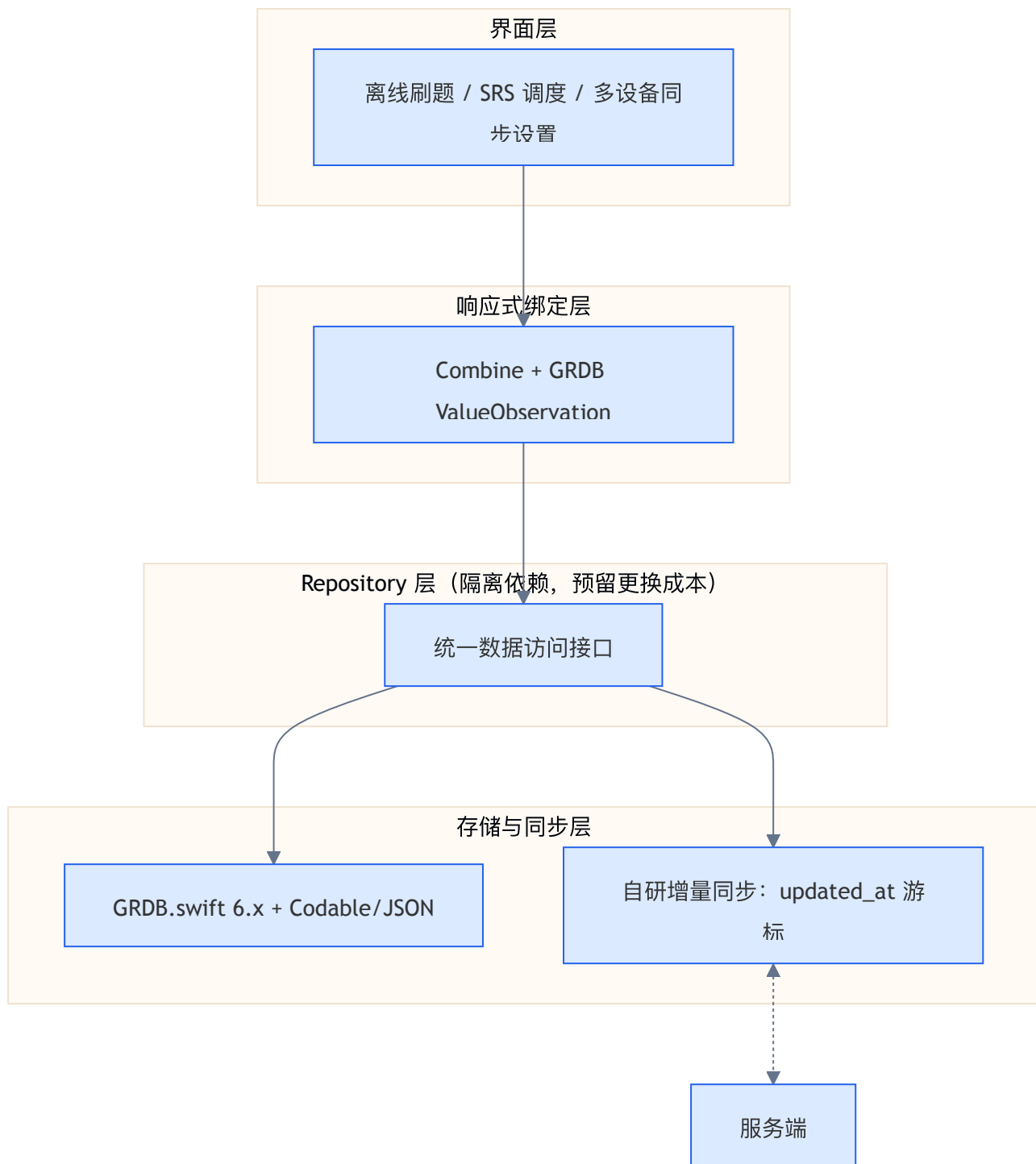
写入卡顿与迁移困难为团队定性结论（未做卡顿率量化），本方案聚焦替换路径与预研证据，不对现状问题做数字外推。

选型结论：四模块成套替换

存储、序列化、同步、绑定四个模块一起换——各自的最优解恰好拼成一套离线优先组合。

模块	选定	落选	一句话理由
本地存储	GRDB.swift 6.x	SwiftData、Core Data（现状）	真机实测五项指标全面占优
序列化	Codable + JSON	Protobuf	-38% 包体收益买不回双端 schema 维护成本
数据同步	自研增量同步 (updated_at 游标)	CloudKit、CRDT	冲突策略可控，不堵死 Android 延伸
响应式绑定	Combine + GRDB ValueObservation	手写通知中心	Swift 官方框架与 GRDB 原生集成

新数据层按四层组织：界面只面对响应式绑定，业务只面对 Repository 层——GRDB 与同步引擎被隔离在最底层，这是对「社区规模小于 Core Data」这条风险的架构级兜底。



存储与同步的可用性不靠推演：前者有真机同口径对比，后者有 POC 与压测——下一节起逐项过证据。

候选对比：官方方案为何落选

落选不是印象分——SwiftData 三条硬伤、CloudKit 两处不可控、Protobuf 一个成本不等式。

SwiftData 与 SwiftUI 亲和度最高，团队评估后放弃：@Model 非 Sendable 限制多线程写入，而离线刷题恰是高并发写场景；iCloud 同步行为黑盒，出问题排障无从下手；迁移无法精细控制——与我们要逃离的痛点同构。

CloudKit 免运维有吸引力，但同步冲突策略不可控，「服务端时间戳优先」这类业务策略无法注入；且无法支持 Android 端的未来规划——同步协议必须握在自己手里。

Protobuf 预研序列化性能占优（包体积减少 38%），但需要维护双端 schema，学习成本高于收益——38% 的体积收益换不来协议层的长期维护负担。

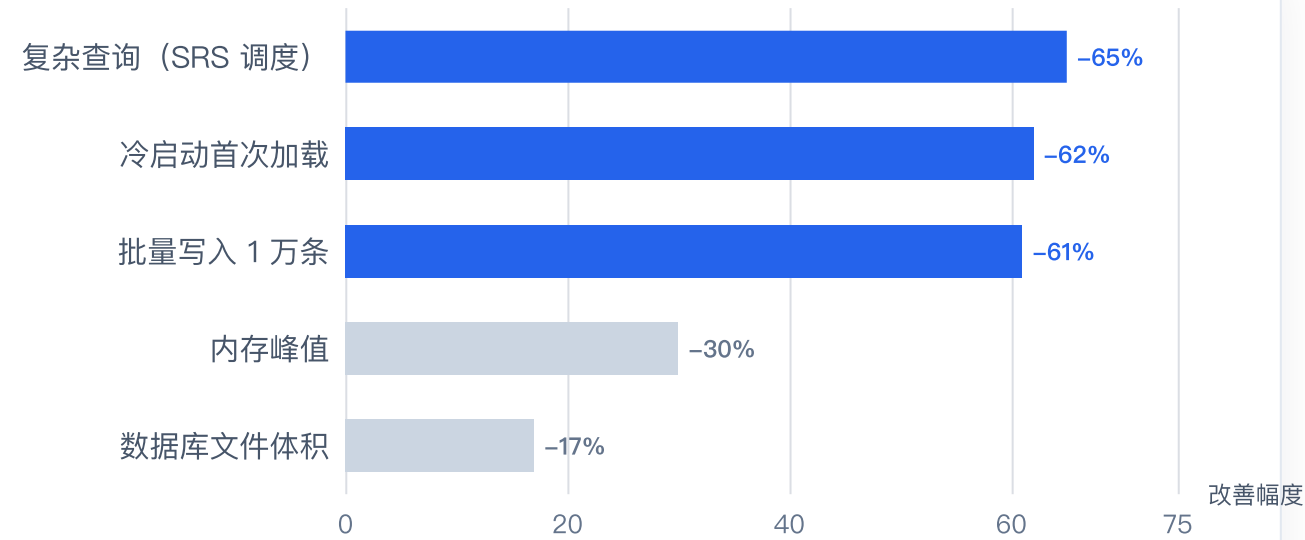
候选	关键短板	可翻案条件
SwiftData	@Model 非 Sendable 限制多线程写入；iCloud 同步行为黑盒；无法精细控制迁移	苹果放开并发模型与迁移控制
CloudKit	同步冲突策略不可控；无法支持 Android 端未来规划	冲突策略可配置且跨端纳入规划
Protobuf	包体积 -38%，但需维护双端 schema，学习成本高于收益	双端团队扩容、schema 工具链成熟
GRDB + 自研同步（本方案）	社区规模小于 Core Data——以 Repository 层隔离兜底；实测证据见下两节	—

官方方案的便利买不回控制权：并发模型、迁移策略、冲突策略三处控制权，都在「换框架」这一票里一并拿回。

预研证据·存储性能：五项指标全面改善

真机同口径实测：不是某项赢，是全面赢——三项提速 61—65%，内存峰值 -30%，库体积 -17%。

预研在 iPhone 13 真机（iOS 17.4，Release 构建）上用同一数据集对比 Core Data 与 GRDB：5 万条单词记录加 120 万条学习记录，覆盖冷启动、批量写入、SRS 调度查询、内存与存储体积五个维度。



五项核心指标全面改善：查询 -65%、冷启动 -62%

iPhone 13 真机 · iOS 17.4 · Release · 5 万单词 + 120 万条学习记录 · 用户提供预研报告口径（评审前复核）

指标	Core Data（现状）	GRDB（新方案）	变化
冷启动首次加载	820 ms	310 ms	-62%
批量写入 1 万条	4.6 s	1.8 s	-61%
复杂查询（SRS 调度）	96 ms	34 ms	-65%
内存峰值	210 MB	148 MB	-30%
数据库文件体积	86 MB	71 MB	-17%

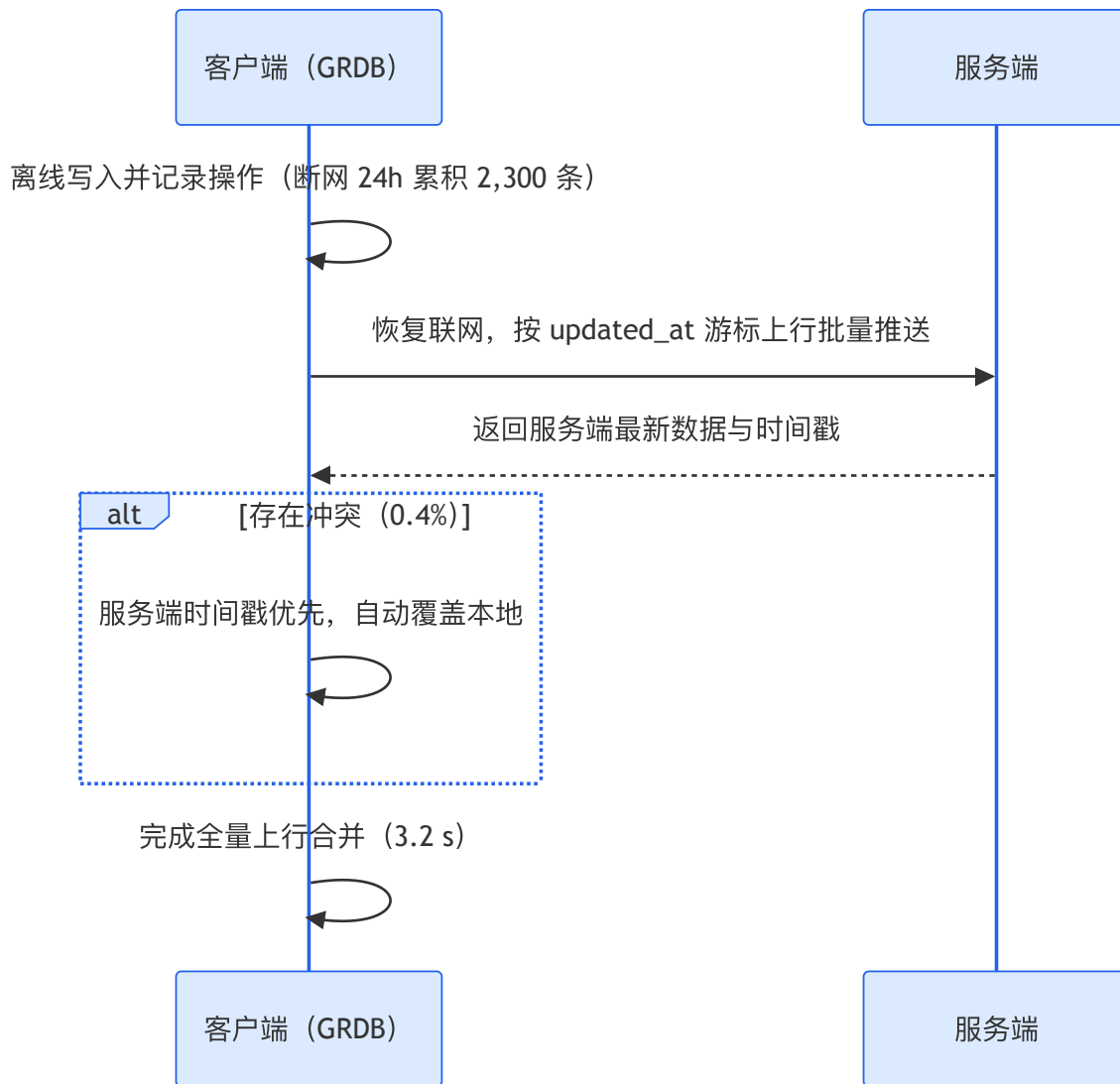
口径说明：变化幅度一律以 Core Data 现状为基数；性能数字转述自预研报告，评审前已与原文核对。

预研证据·同步引擎：POC 证明可用

断网 24 小时累积 2,300 条本地操作，恢复联网 3.2 秒完成全量上行合并；
0.4% 冲突全部自动解决。



同步机制走 `updated_at` 游标的增量协议：本地离线写入时记录操作，联网后按游标批量上行，服务端返回最新数据与时间戳；出现冲突时按「服务端时间戳优先」自动覆盖，无需用户介入。



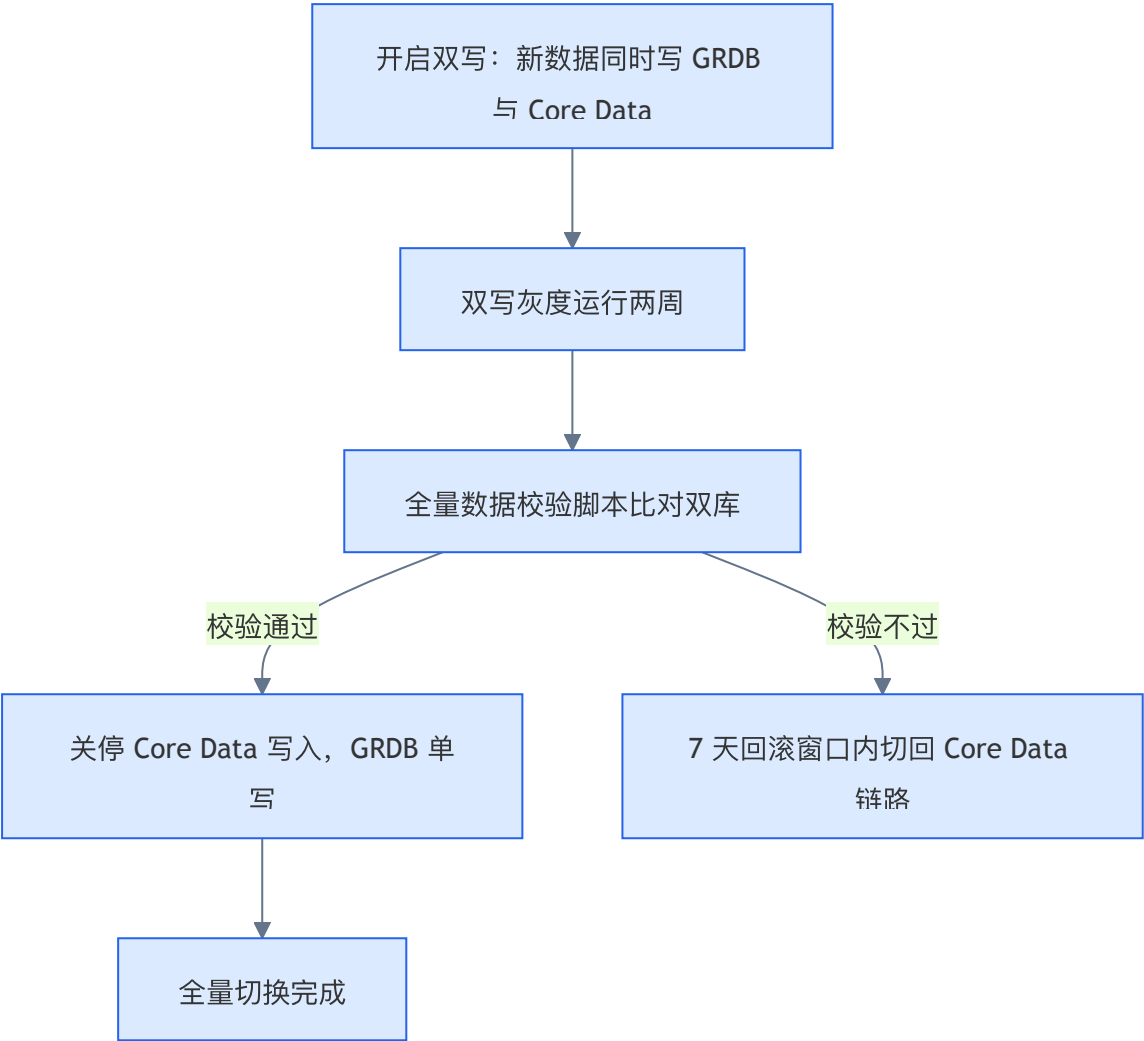
压测口径为服务端同步接口 (模拟 10 万 DAU 场景), 端侧恢复合并表现见上方 POC 指标。以上数字转述自预研报告, 评审前已与原文核对。

风险·迁移·排期： 每条风险都有兜底

GRDB 社区小是真风险，靠 Repository 层隔离兜底；迁移走双写灰度加全量校验，7 天窗口随时可退。

GRDB 社区规模小于 Core Data：一旦停维护或出现重大缺陷，更换成本被 Repository 层锁死在数据访问层之内，界面与业务层零感知——这是选型时已计入的代价。

迁移不赌运气：新数据双写进两套存储，灰度运行两周后用全量数据校验脚本逐条比对，校验通过才关停旧链路；任何时点校验不过，都在 7 天回滚窗口内切回 Core Data。



灰度期内现有功能的回归风险由双写与校验兜底：刷题、SRS 调度走 Repository 层的新链路，数据一致性以脚本比对结果为准，不依赖抽样观察。

Repository 层封装

Q4 第 1-2 周

隔离 GRDB 依赖，为双写与校验脚本打地基

灰度迁移

Q4 第 3-6 周

双写灰度两周 + 全量数据校验脚本，7 天回滚窗口

全量切换

Q4 第 7 周

关停旧链路，GRDB 单写